

CS50 Artificial Intelligence

Lecture 4: Machine Learning — Detailed Study Notes

1. Introduction to Machine Learning

Machine learning (ML) represents a paradigm shift where we do not give a computer explicit instructions on how to perform a task. Instead, we provide the computer with access to data or patterns so that it can learn from them and figure out how to perform the task on its own.

Machine learning is a broad field divided into three primary categories:

1. Supervised Learning

Learning from labeled input-output pairs to find a mapping function.

2. Reinforcement Learning

Learning from trial-and-error experiences using rewards and punishments.

3. Unsupervised Learning

Learning patterns and structure from unlabeled data without explicit feedback.

2. Supervised Learning

In supervised learning, the computer is given a dataset consisting of **input-output pairs**. The goal of the AI is to approximate a true mapping function $f(x) = y$ using an estimated **hypothesis function** $h(x)$. The objective is to make $h(x)$ agree with $f(x)$ as closely as possible.

A. Classification

Classification is the task of mapping inputs into discrete, predefined categories (labels). Examples include:

- Determining whether a banknote is authentic or counterfeit.
- Predicting whether a given day will be rainy or not rainy.

I. Nearest Neighbor Classification

- **Basic Nearest Neighbor:** Given an unlabeled input point, the algorithm classifies it based on the category of its single closest neighbor in the dataset.
- **K-Nearest Neighbor (KNN):** Instead of looking at only one neighbor, the algorithm considers the **K** closest neighbors and assigns the category that receives the majority of votes.
- **Drawbacks:** A naive KNN implementation can be computationally slow because it requires calculating the distance between the query point and every single data point in the dataset (though optimization techniques like pruning or advanced data structures like k-d trees can speed this up).

II. Linear Decision Boundaries & The Perceptron Classifier

To separate categories, we can draw a **decision boundary**. In two dimensions, this boundary can be a line that separates the classes (e.g., dividing rainy days from not rainy days).

- **Linear Separability:** A dataset is *linearly separable* if a single straight line (or hyperplane in higher dimensions) can perfectly separate the two groups. Real-world data is often noisy and not perfectly linearly separable.
- **Mathematical Representation:**
 - Inputs are represented as variables: x_1 (e.g., humidity) and x_2 (e.g., pressure).
 - The boundary is defined as a linear combination of these inputs multiplied by their respective **weights** (w_1 , w_2), plus an additional constant weight (w_0) to shift the function up or down.
 - The decision rule is:

$$\begin{aligned} \text{If } w_1x_1 + w_2x_2 + w_0 \geq 0 &\rightarrow \text{Output 1 (e.g., Rain)} \\ \text{Otherwise} &\rightarrow \text{Output 0 (e.g., No Rain)} \end{aligned}$$

- **Vector Mathematics:**
 - We can construct a **weight vector** $W = [w_0, w_1, w_2]$ and an **input vector** $X = [1, x_1, x_2]$ (where 1 is a dummy input multiplied by w_0).
 - The linear combination is calculated as the **dot product** of W and X :

$$W \cdot X = w_0(1) + w_1x_1 + w_2x_2$$

- The hypothesis function $h(X)$ outputs 1 if $W \cdot X \geq 0$, and 0 if $W \cdot X < 0$.

III. The Perceptron Learning Rule

To find the correct weights, the AI starts with random weights and updates them iteratively as it learns from each data point (X, Y) :

Update Formula: For each weight w_i :

$$w_i \leftarrow w_i + \alpha(y - h(X))x_i$$

- y : Actual label (1 or 0)
- $h(X)$: Predicted label (1 or 0)
- α : **Learning rate**, which determines how quickly the weights are updated

Update Logic:

- If prediction is correct ($y = h(X)$), then $(y - h(X)) = 0$, so weights remain unchanged.
- If actual is 1 but estimate is 0 ($y > h(X)$), we add a positive value to increase weight.
- If actual is 0 but estimate is 1 ($y < h(X)$), we subtract a value to decrease weight.

IV. Hard vs. Soft Thresholding

- **Hard Threshold:** Used by standard Perceptron. Outputs only 0 or 1, leaving no room for uncertainty.
- **Soft Threshold (Logistic Regression):** Transforms the hard step function into a smooth logistic function. Outputs continuous values between 0 and 1 (e.g., 0.7), representing probability/confidence.

V. Support Vector Machines (SVM)

- **Motivation:** Infinite lines can separate two classes, but some are too close to data points and generalize poorly.
- **Maximum Margin Separator:** SVM maximizes the distance to the closest points of either class (these critical points are called **support vectors**).
- **Higher Dimensions:** Uses hyperplanes in multi-dimensional space to find non-linear decision boundaries for complex datasets.

B. Regression

Regression is a supervised learning task where the output is a **continuous real-valued number** rather than a discrete category.

- *Example:* Predicting a company's sales revenue based on its advertising budget.
- **Linear Regression:** Finds a line of best fit that approximates the relationship between inputs and outputs.

C. Evaluating Hypotheses & Optimization

We evaluate performance by minimizing a **Loss Function**, which quantifies model error.

Loss Function	Type	Formula / Definition	Characteristics
0-1 Loss	Classification	$Loss = 0$ if Actual = Predicted, else 1	Simple accuracy count.
L1 Loss	Regression	$Loss = Actual - Predicted $	Sum of absolute differences. Robust to outliers.
L2 Loss	Regression	$Loss = (Actual - Predicted)^2$	Penalizes large errors & outliers more severely.

D. Overfitting & Regularization

- **Overfitting:** Occurs when a model fits the training data too closely (capturing noise/outliers), failing to generalize to new unseen data.
- **Regularization:** Prevents overfitting by penalizing complex hypotheses (favoring Occam's Razor):

$$Cost(h) = Loss(h) + \lambda \times Complexity(h)$$

Where λ (Lambda) controls penalty severity.

E. Model Validation Techniques

1. **Holdout Cross-Validation:** Splits data into a *training set* and a *testing set*.
2. **K-Fold Cross-Validation:** Splits data into K equal folds. Runs K experiments using 1 fold for testing and $K-1$ for training in each iteration, then averages performance.

3. Reinforcement Learning

In reinforcement learning, the agent learns from **experience** through rewards and punishments, rather than a pre-labeled dataset.

A. Core Concepts

- **Agent:** The decision-making entity.
- **Environment:** The world in which the agent operates.
- **State (s):** Current configuration of agent and environment.
- **Action (a):** Choice made by the agent.
- **Reward (R):** Feedback signal (positive reinforcement or negative punishment).

B. Markov Decision Process (MDP)

A formal framework for decision-making consisting of:

- Set of states (S) and available actions ($A(s)$).
- **Transition Model $P(s' | s, a)$:** Probability of reaching state s' given state s and action a .
- **Reward Function $R(s, a, s')$:** Reward received for transitioning.

C. Q-Learning

Learns a function $Q(s, a)$ estimating total expected reward for taking action a in state s .

I. Initialization

$$Q(s, a) = 0 \text{ \&quad; } \forall s, a$$

II. Update Rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha [\text{New Estimate} - Q(s, a)]$$

where $\text{New Estimate} = R + \gamma \max_{a'} Q(s', a')$

- α (Learning Rate): Balances new info vs. old knowledge.
- γ (Discount Factor): Values immediate vs. long-term future rewards.

III. Exploration vs. Exploitation (ϵ -Greedy)

- **Exploitation:** Select best known action ($a = \operatorname{argmax}_a Q(s, a)$).
- **Exploration:** Try new actions to discover better rewards.
- **ϵ -Greedy Strategy:** Exploit with probability $1 - \epsilon$, explore randomly with probability ϵ .

IV. Function Approximation

Used for massive state spaces (e.g., Chess) where exact lookup tables are intractable. Estimates $Q(s, a)$ using state features.

4. Unsupervised Learning

Learning without human labels or explicit feedback. The AI identifies intrinsic patterns or groupings in data.

A. Clustering & K-Means

Groups similar data points together. **K-Means Clustering** follows an iterative 5-step process:

1. **Initialization:** Randomly place K cluster centroids.
2. **Assignment:** Assign each data point to its nearest centroid.
3. **Recentering:** Move each centroid to the mean position of its assigned points.
4. **Iteration:** Repeat steps 2 and 3.
5. **Convergence:** Stop when centroids stabilize and assignments no longer change.